

PCCST503 – Machine Learning

User Manual – Safe Semantic Planner

Student: Nabeel T

Register No: LTCR24CS075

Algorithm: D* Lite

This manual explains how to compile, run, modify, and demonstrate the Safe Semantic Planner implemented for Assignment 1.

1. Purpose

This program demonstrates safe path planning in a finite directed graph. It finds a route from an initial state to a goal while avoiding states marked as bad. It also reports cost, safety margin, reliability, explored states, and planning time.

2. Requirements

A C++17-compatible compiler is recommended. Examples include GCC, Clang, and Visual Studio with C++17 support. No third-party libraries are required.

3. Compile

Linux/macOS with GCC or Clang:

```
g++ -std=c++17 -O2 PCCST503_Safe_Semantic_Planner.cpp -o planner
```

Windows (MinGW):

```
g++ -std=c++17 -O2 PCCST503_Safe_Semantic_Planner.cpp -o planner.exe
```

4. Run

Linux/macOS: ./planner

Windows: planner.exe

5. Demonstration

The included main() creates small Cartesian graph examples. Bad states are marked explicitly, so transitions entering those states are rejected. The program demonstrates safe route planning and dynamic changes such as making a transition unavailable, changing the goal, and adding a shortcut transition.

6. Output Fields

Success – whether a safe route was found.

State path – ordered state IDs from start to goal.

Transition path – IDs of transitions used.

Total cost – sum of transition costs.

Minimum safety distance – smallest Euclidean distance from a visited state to the nearest bad state.

Cumulative reliability – product of transition reliabilities along the path.

Explored states – number of states processed by the search.

Planning time – elapsed time for a planning call.

7. Modifying the Input

Add or edit State entries in the states vector. Each state needs an ID and a Cartesian embedding, for example {10, {2.0, 3.0}}. Add bad-state IDs to badStates. Add directed transitions using from, to, cost, safety, reliability, and available. To disable an edge dynamically, set its available field to false.

8. Dynamic Replanning

After changing goalState, badStates, transitions, or transition availability, run the planner again so the new environment is reflected. A fully incremental D* Lite implementation can preserve g/rhs data and update only affected vertices; this is useful for larger real-time environments.

9. Interpreting Safety

Safety is a hard constraint, not merely a preference: bad states are never expanded. The minimum safety distance is a separate quality measure. If two candidate paths have the same cost, the implementation can prefer the one whose closest point to a bad state is farther away.

10. Troubleshooting

If the result is **Success: NO**, check that the initial and goal states exist, neither is bad, transitions are directed as intended, and at least one available route connects them. If compilation fails, ensure C++17 support is enabled. If safety distance is infinity, verify that badStates is intentionally empty.

11. Suggested Demonstration Checklist

1. Compile and run the program.
2. Show the initial safe route.
3. Point out that bad states are avoided.
4. Disable a transition and show replanning.
5. Change the goal and show the revised route.
6. Add a shortcut transition and show the improved route.
7. Explain total cost, minimum safety distance, explored states, reliability, and planning time.